

CPT204

1. Creating Arrays

三种方法：

```
int[] arr = {1, 2, 3, 4, 5};
```

```
int[] arr = new int[] {1, 2, 3, 4, 5};
```

```
int[] arr = new int[5];
```

2. Default values

int / double / byte / short / long / float -> 0

char -> '\u0000'

boolean -> false

reference type -> null

3. Static variable

是“属于类”的变量

Static variable 是所有对象共享的

```
class Circle {  
    static int count = 0;  
    double radius;  
    Circle(double r) {  
        radius = r;  
        count++;  
    }  
}
```

然后：

```
Circle c1 = new Circle(2.0);
```

```
Circle c2 = new Circle(5.0);
```

```
Circle c3 = new Circle(10.0);  
  
System.out.println(Circle.count);
```

输出: 3

为什么是 3? 因为每创建一个 Circle 对象, 构造方法里都会执行: count++;但是 count 是 static 的, 所有对象共享同一个 count。所以: 创建 c1 后: count = 1 创建 c2 后: count = 2 创建 c3 后: count = 3

Eg. D C

In-Class Quiz 1: Static vs Non-Static

- What is the correct code to print the value of x ?

```
1 public class InClassQuiz {  
2  
3     int x = 5;  
4     static int y = 10;  
5  
6     public static void main(String[] args) {  
7  
8         System.out.println( );  
9  
10    }  
11 }
```

- ☐ x
- ☐ this.x
- ☐ InClassQuiz.x
- ☐ new InClassQuiz().x

In-Class Quiz 2: Static vs Non-Static

- What is the correct code to print the value of y ?

```
1 public class InClassQuiz {  
2  
3     int x = 5;  
4     static int y = 10;  
5  
6     public static void main(String[] args) {  
7  
8         System.out.println( );  
9  
10    }  
11 }
```

- ☐ y
- ☐ this.y
- ☐ InClassQuiz.y
- ☐ new InClassQuiz().y

注意: No Default values for local variables, Data fields have default values

No Default values for local variables

Java assigns no default value to a **local variable** inside a method.

```
public class Test {  
    public static void main(String[] args) {  
        int x; // x has no default value  
        String y; // y has no default value  
        System.out.println("x is " + x);  
        System.out.println("y is " + y);  
    }  
}
```

Compilation errors: the variables are not initialized

Default values for Data Fields

- Data fields have default values

- Example:

```
public class Student {
    String name; // name has default value null
    int age; // age has default value 0
    boolean isScienceMajor; // isScienceMajor has default value false
    char gender; // c has default value '\u0000'
}

public class Test {
    public static void main(String[] args) {
        Student student = new Student();
        System.out.println("name? " + student.name);           // null
        System.out.println("age? " + student.age);             // 0
        System.out.println("isScienceMajor? " + student.isScienceMajor); // false
        System.out.println("gender? " + student.gender);       //
    }
}
```

Note: If a data field of a reference type does not reference any object, the data field holds a special literal value: **null**.

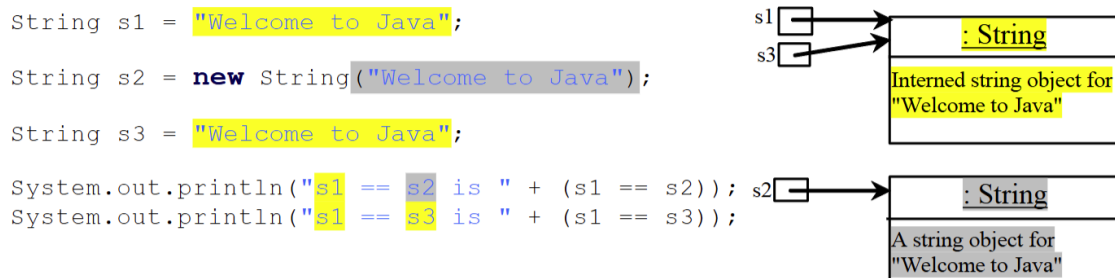
9. 局部变量 vs 成员变量

这是这页最重要的对比。

对比点	Local variable 局部变量	Data field variable 成员变量
声明位置	方法、if、for 等代码块内部	类里面，方法外面
作用域	从声明处开始，到所在 block 结束	整个类
是否有默认值	没有，必须手动赋值	有默认值
生命周期	方法/代码块执行结束就消失	随对象或类存在
例子	<code>int x = 5;</code> 在方法里面	<code>int age;</code> 在类里面

4.Immutable和mutable

- **String interning** is a method of storing **only one copy** of each distinct compile-time constant/explicit string in the source code stored in a string intern pool (e.g., s1 and s3).
- If we use the **new** operator, then a new object is created in heap (e.g., s2).



displays: s1 == s3 is **true**

s1 == s2 is **false**

"Welcome to Java" 这个字符串字面量会放在字符串常量池中。第二次写 "Welcome to Java" 时，Java 发现常量池里已经有 "Welcome to Java"，就不会再创建新的，而是让 s2 也指向同一个 "Welcome to Java"。但是注意：数组是没有这样的操作的，因为数组可变不会放在常量池，字符串是不可变的。

```
if (s1.equals(s2)) { // true
    // s1 and s2 have the same contents
}

if (s1 == s2) { // false
    // s1 and s2 have different references
}
```

5. Abstract相关

An abstract method can only be contained in an abstract class. 抽象方法只能在abstract 类中

In an abstract subclass extended from an abstract super-class, we can choose: to implement the inherited abstract methods OR to postpone the constraint to implement the abstract methods to its nonabstract subclasses.

abstract可以有instance method，但是interface没有！！！！

6. 继承相关

(1)父类构造方法不会被子类继承，因为构造方法的名字必须和类名一样

(2)创建子类对象时，父类构造方法一定会先被调用。

(3)子类构造方法如果不写 `super(...)`，默认在子类构造方法第一行加`super()`；但是 `super()` 是调用父类无参构造方法，所以父类必须真的有一个无参构造方法。但是如果父类没有无参构造方法，只有有参构造方法，那么子类必须手动写 `super(参数)`。否则父类没有无参构造方法，就会编译报错。

(4) `this(...)` 和 `super(...)` 不能同时直接出现在同一个构造方法的第一行。`this(...)` 表示调用本类的另一个构造方法，`super(...)` 表示调用父类构造方法。它们都要求必须写在构造方法第一行，所以不能同时直接写在同一个构造方法中。

(5)构造方法链 Constructor Chaining: Java 会先一路找到最上层父类构造方法，然后从最上层父类开始一层一层执行回来。

7.overloading和overriding

方法名相同，参数列表不同叫overloading

方法名一样，参数列表一样，返回类型也一样叫overriding

编译时先看左边 reference type 有没有对应方法；运行时再看右边 actual object 里有没有 override 的方法。

eg

```
class B {
    public void p(double i) {
        System.out.println(i * 2);
    }
}

class A extends B {
    public void p(int i) {
        System.out.println(i);
    }
}

// 执行部分
B a = new A();
a.p(10);
```

`a` 的 reference type 是 `B`，所以 `a.p(10)` 编译时只能在 `B` 中找方法。`B` 中有 `p(double)`，而 `10` 可以自动转换成 `10.0`，所以选择 `p(double)`。运行时 actual object 是 `A`，但 `A` 没有 override `p(double)`，它只有 overload 的 `p(int)`，所以最终调用 `B` 的 `p(double)`，输出 `20.0`。

左边 `B` 叫 reference type，也叫 declared type / compile-time type。

右边 `new A()` 叫 actual object，也叫 object type / runtime type。

8.Method Matching vs. Binding

- For overloaded methods, the compiler finds a matching method according to parameter type, number of parameters, and order of the parameters at compilation time.
- For overridden methods, the Java VirtualMachine dynamically binds the implementation of the most specific overridden method implementation at runtime.

Overloading: 编译时决定, 看参数。

Overriding: 运行时决定, 看实际对象。

注意: 只有“可以被 **overriding** 的实例方法”才有动态绑定。而 `static`、`private`、`final` 方法都不是普通的 dynamic binding 情况。

比如: `Parent p = new Child();`

`p.test();` //test是一个 `static/private/final` 方法

这里test()方法不能被override, 因此调用的都是左边Parent类里面的方法。

如果 test() 是普通实例方法, 并且 Child 重写了 test(), 那么运行时调用 Child 里的 test()。

但是如果 test() 是 static、private 或 final 方法, 就不是普通的动态绑定情况:

1. static 方法属于类, 不属于对象。它不能被 override, 只能被隐藏 method hiding。因此 `Parent p = new Child(); p.test();` 会根据左边引用类型 Parent 调用 `Parent.test()`。
2. private 方法不能被子类继承, 也不能被 override。如果 test() 是 private, 那么在类外部通常不能直接写 `p.test()`, 因为 private 方法不可访问。如果是在 Parent 类内部调用 private test(), 调用的一定是 Parent 自己的 test()。
3. final 方法可以被继承, 但不能被 override。因此 Child 不能重新定义同样签名的 test()。所以 `Parent p = new Child(); p.test();` 最终只能调用 Parent 里的 final test()。

总结: 普通实例方法看右边实际对象, static 方法看左边引用类型, private 方法只属于本类, final 方法不能被重写。

9.equality问题

In Java, **reference equality(==)** checks if two variables refer to the exact same object in memory, while **content equality(equals)** checks if two objects have the same state or content, as defined by the equals() method

注意:

`Object o1 = new Object();`

`Object o2 = new Object();`

`System.out.print((o1 == o2) + "and" + (o1.equals(o2)));`

答案是false and false.

10. Primitive and reference

A primitive variable stores the actual values of primitive data types directly, while a reference variable stores the address of an object in memory.

11. Generics

The key benefit of generics is to enable errors to be detected at compile time rather than at runtime.

Most important advantage: If you attempt to use the class or method with an incompatible object, a compile error occurs

Generic types must be reference types! You cannot replace a generic type with a primitive type such as int, double, or char.

No Casting for Get and Unboxing

```
ArrayList<Double> list = new ArrayList<Double>();
list.add(5.5); // 5.5 is automatically boxed/
               // converted to new Double(5.5)
list.add(3.0); // 3.0 is automatically boxed/
               // converted to new Double(3.0)

Double doubleObject = list.get(0);
// No casting is needed

• Also automatic Unboxing:
double d = list.get(1);
// Unboxing: Automatic conversion to double
• This is a property of numeric wrapper classes.
```

Declaring Generic Classes

GenericStack.java

```
public class GenericStack<E> {
    private java.util.ArrayList<E> list = new java.util.ArrayList<E>();
    public int getSize() {
        return list.size();
    }
    public E peek() {
        return list.get(getSize() - 1);
    }
    public void push(E o) {
        list.add(o);
    }
    public E pop() {
        E o = list.remove(getSize() - 1);
        return o;
        // OR just: return list.remove(getSize() - 1);
    }
    public boolean isEmpty() {
        return list.isEmpty();
    }
    @Override // Java annotation: also used at compile time to
    public String toString() { // detect override errors
        return "stack: " + list.toString();
    }
}
```

15

(c) Paul Fodor (CS Stony Brook) & Pearson

Generic Static Methods

- To declare a generic method, you place the generic type immediately after the keyword **static** in the method header:

```
public class GenericMethods1 {
    public static <E> void print(E[] list) {
        for (int i = 0; i < list.length; i++)
            System.out.print(list[i] + " ");
        System.out.println();
    }
    public static void main(String[] args) {
        String[] s3 = {"Hello", "again"};
        GenericMethods1.<String>print(s3);
        // OR simply:
        print(s3);
    }
}
```

7

(c) Paul Fodor (CS Stony Brook) & Pearson

如果一个类是泛型类（例如 `class MyClass<T>`），这个 `<T>` 只有在 `new` 对象的时候才能确定。而 `static` 方法不需要 `new` 对象就能直接通过类名调用。因此，静态方法绝对无法使用类名上声明的那个泛型，静态方法如果想用泛型，就必须在自己的方法签名里独立声明 `<E>`。它与类是不是泛型类毫无关系。

Bounded Generic Types

- A generic type can be specified as a subtype of another type:

- consider `Circle` and `Rectangle` extend `GeometricObject`

```
public class GenericMethods2 {
    public static <E extends GeometricObject> boolean
        equalArea(E object1, E object2) {
        return object1.getArea() == object2.getArea();
    }
    public static void main(String[] args ) {
        Rectangle rectangle = new Rectangle(2, 2);
        Circle circle = new Circle(2);
        System.out.println("Same area? " +
            equalArea(rectangle, circle));
    } // Note: see https://www3.cs.stonybrook.edu/~pfodor/courses/cse160.html
    // for GenericObject, Circle and Rectangle
}
```

```
public static <E extends Comparable<E>>
    void genericSelectionSort(E[] list) {
```

- The generic type `<E extends Comparable<E>>` has two meanings:
 - First, it specifies that **E** must be a subtype of `Comparable`
 - Second, it specifies that the elements to be compared are of the **E** type as well

原始类型 (Raw Type) 与向后兼容性 (Backward Compatibility) :

Raw Type and Backward Compatibility

- A generic class or interface used without specifying a concrete type, called a raw type, enables backward compatibility with earlier versions of Java

- In JDK1.5 and higher, the *Raw type*:

```
ArrayList list = new ArrayList();
```

is roughly equivalent to:

```
ArrayList<Object> list = new ArrayList<Object>();
```

- Therefore, all programs written in previous JDK versions are still executable.
- **Note: Backward compatibility:** a technology is backward compatible if the input developed for an older technology can work with the newer technology:
 - Note: Python 3.X broke backward compatibility to Python 2.X
 - print 1

23

works in Python 2, but not Python 3

(c) Paul Fodor (CS Stony Brook) & Pearson

Wildcards

- A problem to be solved:
 - **Integer** is a subclass of **Number**, but **GenericStack<Integer>** is not a subclass of **GenericStack<Number>**
 - You can use unbounded wildcards, bounded wildcards, or lower-bound wildcards to specify a range for a generic type:
 - **?** unbounded wildcard: any class
 - **? extends T** bounded wildcard: any subclass of T (including T)
 - **? super T** lower bound wildcard: any superclass of T (including T)
 - **<? extends Number>** is a wildcard type that represents **Number** or a subtype of **Number**
 - **GenericStack<Integer>** is a subclass of **GenericStack<? extends Number>**

```
GenericStack<? extends Number> stack1 = new GenericStack<Double>(); // 正确
```

```
GenericStack<? extends Number> stack2 = new GenericStack<Number>(); // 正确
```

通配符（**?**）是用来声明“变量”或“方法参数”的（即写在左边），而 **new** 创建真实对象时（即写在右边），尖括号里必须写具体的、确定的某个类。

Erasure and Restrictions on Generics

- Generics are implemented in Java using an approach called type erasure:
 - The compiler uses the generic type information to compile the code, but erases it afterwards
 - This approach enables the generic code to be backward-compatible with the legacy code that uses raw types
 - So the generic information is not available at run time
 - Once the compiler confirms that a generic type is used safely, it converts the generic type back to a raw type **Object**
 - For example, the compiler checks whether the following code in (a) uses generics correctly and then translates it into the equivalent code in (b) for runtime use. The code in (b) uses the raw type **Object**

```
ArrayList<String> list = new ArrayList<String>();  
list.add("Oklahoma");  
String state = list.get(0);
```

(a)

```
ArrayList list = new ArrayList();  
list.add("Oklahoma");  
String state = (String)(list.get(0));
```

(b)

编译器在编译代码时，会利用你写的泛型信息（比如 `<String>`）进行严格的类型安全检查，但是检查完之后，它就会把这些泛型信息彻底擦除（erases it afterwards）。在运行时（Run time）：泛型信息完全不可用（not available）。在 JVM 看来，全世界只有一种 `ArrayList`，它里面装的全部都是最顶层的原始类型—— `Object`

- The generic class is shared by all its instances regardless of its actual generic type
`GenericStack<String> stack1 = new GenericStack<String>();`
`GenericStack<Integer> stack2 = new GenericStack<Integer>();`

- Although `GenericStack<String>` and `GenericStack<Integer>` are two types, but there is only one class `GenericStack` loaded into the JVM

- These are **true**:

```
System.out.println(stack1 instanceof GenericStack);  
System.out.println(stack2 instanceof GenericStack);
```

- These are compile errors:

```
System.out.println(stack1 instanceof GenericStack<String>);  
System.out.println(stack2 instanceof GenericStack<Integer>);
```

since `GenericStack<String>` and `GenericStack<Integer>` are not stored as separate classes in the JVM, using them at runtime makes no sense

运行期不分家： `ArrayList<String>` 和 `ArrayList<Integer>` 在运行期的 Class 对象是同一个（都是 `ArrayList.class`）。**instanceof 后面禁止带尖括号：**除了无界通配符（如 `instanceof ArrayList<?>`）这种极特殊情况外，**instanceof 后面绝对不允许写具体的泛型类型**（如 `instanceof ArrayList<String>` 必报错）。

Restrictions on Generics

Restriction 1: Cannot Create an Instance of a Generic Type(i.e., new E()).

For example, the following statement is wrong: E object = new E(); The reason is that new E() is executed at runtime, but the generic type E is not available at runtime

Restriction 2: Generic Array Creation is Not Allowed (i.e., new E[100]).

For example, the following statement is wrong: E[] elements = new E[capacity];

Restriction 3: Since all instances of a generic class have the same runtime class, the static variables and methods of a generic class are shared by all its instances. Therefore, it is illegal to refer to a generic type parameter for a class in a static method, field, or initializer. 在静态 (static) 成员中，绝对禁止使用类上声明的泛型参数（如 <E>），要定义自己的泛型参数（和类上的不一样，比如T）

Restriction 4: Exception Classes Cannot be Generic.

12.Interface

An interface is a class-like construct that **contains only abstract methods and constants**.

In an interface: All data fields are public static final, All methods are public abstract.

- These modifiers can be omitted:

<pre>public interface T1 { public static final int K = 1; public abstract void p(); }</pre>	Equivalent	<pre>public interface T1 { int K = 1; void p(); }</pre>
--	------------	--

- A constant defined in an interface can be accessed using InterfaceName.CONSTANT_NAME, for example: T1.K

Like an abstract class, you cannot create an instance from an interface using the new operator.

Marker Interface: is an empty interface (does not contain constants or methods), but it is used to denote that a class possesses certain desirable properties to the compiler and the JVM. 内部既没有定义任何常量，也没有定义任何抽象方法。

Shallow vs. Deep Copy

浅拷贝：如果类的某个属性是引用类型（对象），克隆时只会复制这个属性的内存地址（引用），而绝对不会复制这个属性的真实内容。

深拷贝：不仅复制表面，连内层所有引用的子对象也要一并执行真正的 `new` 复制。

For *deep copying*, we can override the clone method with custom object creation:

```
public class House implements Cloneable {
    ...
    public Object clone() { // deep copy
        try {
            House h = (House) (super.clone());
            h.whenBuilt = (Date) (whenBuilt.clone());
            return h;
        } catch (CloneNotSupportedException ex) {
            return null;
        }
    }
    ...
}
```

Interface and Abstract

Interfaces vs. Abstract Classes

- In an interface, the data fields must be constants; an abstract class can have variable data fields
- Interfaces don't have constructors; all abstract classes have constructors
- Each method in an interface has only a signature without implementation (i.e., only abstract methods); an abstract class can have concrete methods

	Variables	Constructors	Methods
Interfaces	All variables must be <u>public static final</u>	No constructors. An interface cannot be instantiated using the new operator.	All methods must be <u>public abstract methods</u>
Abstract classes	No restrictions	Constructors are invoked by subclasses through constructor chaining . An abstract class cannot be instantiated using the new operator.	No restrictions.

13.final相关

A final method cannot be overridden by its subclasses

A final class cannot be extended:

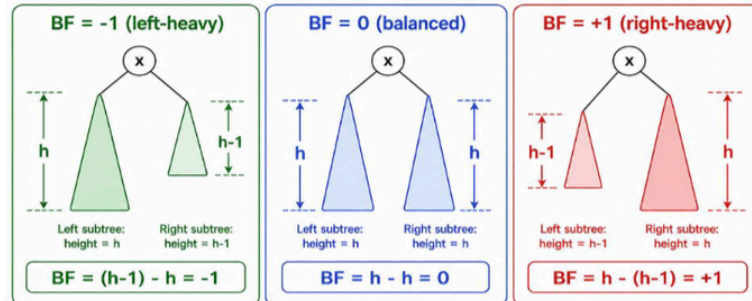
14.AVL Trees and BST

注意：我们学的高度差的定义是**右-左**

BF = 0: both subtrees have the same height (perfectly balanced).

BF = -1: the left subtree is one level taller. The node is still well-balanced, but slightly **left-heavy**.

BF = +1: the right subtree is one level taller. The node is still well-balanced, but slightly **right-heavy**.



4

There are four possible rotations for four possible imbalance:

- **Left-heavy Left-heavy imbalance** => **LL rotation**
- **Right-heavy Right-heavy imbalance** => **RR rotation**
- **Left-heavy Right-heavy imbalance** => **LR rotation**
- **Right-heavy Left-heavy imbalance** => **RL rotation**

AVL的插入和删除和BST相同，本质就多了一个高度的限制

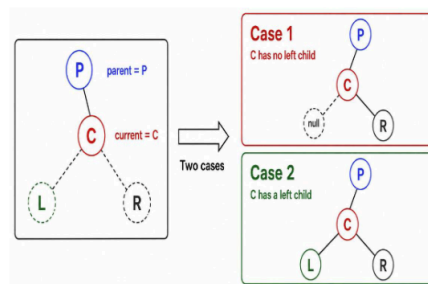
Deleting Elements in a Binary Search Tree

Step 1: Find the node to delete and set its parent.

- current points to the target node
- parent points to the parent node of current

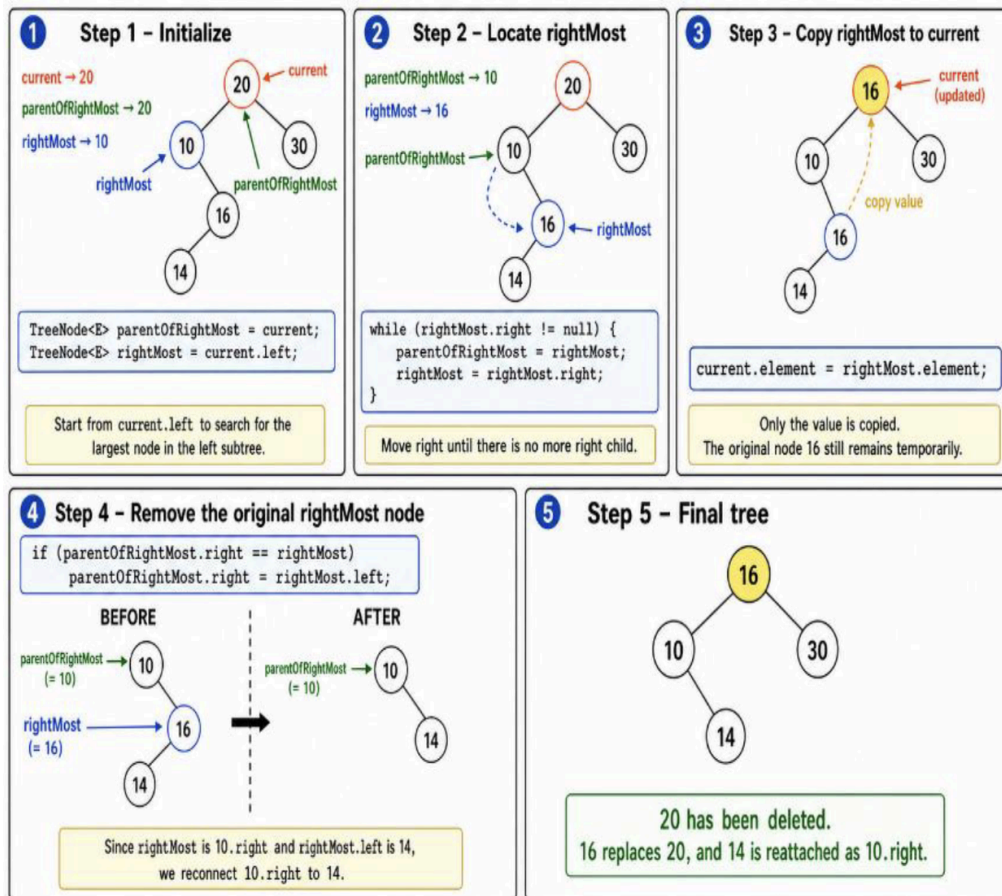
Step 2: Handle one of two cases:

- Case 1: current has no left child
- Case 2: current has a left child



中序遍历的顺序里，排在被删除节点前一个的节点放到被删除节点上

eg.



15.Hashing

A hashing collision occurs when two different keys are mapped to the same index in a hash table.

There are two ways for handling collisions: **open addressing** and **separate chaining**

1. Open addressing is the process of finding an open location in the hash table in the event of a collision. Open addressing has several variations: linear probing, quadratic probing and double hashing

2. Separate chaining places all entries with the same hash index into the same location in a list

Linear Probing会导致clusters (Primary Clustering) . Quadratic Probing会导致secondary clustering.

Load Factor and Rehashing在讲义最后， We should maintain the load factor **under 0.5 for the open addressing schemes** and **under 0.9 for the separate chaining scheme**. If the load factor goes beyond the threshold , we increase the hashtable size and reload the entries into a new larger hash table ->this is called rehashing

16.EDI

1.Equality: 平等

简单来说, **Equality** 在 OOP 项目中的意思是: 每个开发者都应该有平等参与项目的机会。

不应该因为一个人的性别、种族、身体情况、语言背景、技术水平差异等因素, 就让他/她无法参与开发、表达意见或者承担重要任务。

2.Diversity: 多样性

意思是: **Diversity** 强调不同背景、经验和观点的人一起合作, 可以让问题解决方式更丰富、更有创造力。

在软件开发中, 多样性不是只指性别或文化背景, 也可以包括: 不同的编程经验、不同的思维方式、不同的算法理解、不同的测试习惯、不同的用户视角。

3.Inclusion: 包容性

意思是: **Inclusion** 不只是让不同的人“存在”于团队中, 而是要让他们真正被尊重、被倾听、被重视。

这和 Equality 有区别。Equality 关注的是“有没有平等机会”。Inclusion 关注的是“即使有机会, 大家是否真的能舒服地参与”

Practical Ways to Incorporate EDI into OOP Practices

Adopt Inclusive Coding Practices:

- Use inclusive language in code, comments, and documentation. Avoid terms or practices that could inadvertently alienate or exclude individuals.

Implement Accessible Design Patterns:

- Ensure that the software designs you build are accessible to a wide audience. Incorporate accessibility features such as customizable UI options, voice recognition, and high-contrast themes.

Encourage Collaboration Across Diverse Teams:

- Actively seek out team members from diverse backgrounds and experiences. This could mean promoting women in tech, or working with teams that include people from different ethnicities and cultures.

17.PriorityQueue

在 Java 里, 默认 **PriorityQueue** 是小顶堆 min-heap, 也就是: 数值越小, 优先级越高。

Integer默认按照从小到大排序

String默认按照字典序排序

Character字符顺序靠前的优先

18.Itearable

`Iterable` 表示一个对象可以被遍历。它要求类实现 `iterator()` 方法，返回一个 `Iterator`。`Iterator` 通过 `hasNext()` 和 `next()` 一个一个访问元素，同时不需要暴露集合内部的存储结构。

Obtaining the Iterator

```
Collection<String> collection = new ArrayList<>();
collection.add("New York");
collection.add("Atlanta");

Iterator<String> iterator = collection.iterator();

while (iterator.hasNext()) {
    System.out.print(iterator.next().toUpperCase() + " ");
}
```

Output:

```
NEW YORK ATLANTA

It works, but writing the while loop is a lot of manual work.
```

19.Comparable vs Comparator

7. Comparable vs Comparator 对比

对比点	Comparable	Comparator	
位置	写在类内部	写在类外部	
方法	<code>compareTo()</code>	<code>compare()</code>	
参数数量	一个参数	两个参数	
用途	定义自然顺序	定义自定义顺序	
是否需要修改类	需要修改类	不一定需要修改类	
常见用法	<code>Collections.sort(list)</code>	<code>Collections.sort(list, comparator)</code>	

```
</> Java

class ScoreComparator implements Comparator<Student> {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.score - s2.score;
    }
}
```

使用:

```
</> Java

Collections.sort(students, new ScoreComparator());
```

Comparable

```
</> Java

class Student implements Comparable<Student> {
    @Override
    public int compareTo(Student other) {
        return this.score - other.score;
    }
}
```

调用:

```
</> Java

Collections.sort(students);
```

这里 `sort` 会自动使用 `Student` 里的 `compareTo()` 。

20.两种图算法比较

Prim v.s. Dijkstra

Aspect	MST / Prim’s Algorithm	Dijkstra’s Algorithm
Main Goal	Connect all vertices with the minimum total edge weight.	Find the shortest path from one source vertex to all other vertices.
Key Question	“What is the cheapest way to connect all vertices?”	“What is the shortest distance from the source to each vertex?”
Meaning of cost[v]	The cheapest edge weight connecting v to the current tree T.	The current shortest known distance from the source to v.
What Gets Chosen Each Step	The vertex outside T that is cheapest to connect to T .	The vertex outside T with the smallest distance from the source .
Final Output	A minimum spanning tree.	A shortest-path tree from the source.
Best Use Case	Building a low-cost network connecting all nodes.	Finding shortest routes from one starting point.

21.Java 面向对象四大特征

特征	英文	核心意思
封装	Encapsulation	隐藏数据，保护属性
继承	Inheritance	子类复用父类代码
多态	Polymorphism	父类引用指向子类对象
抽象	Abstraction	定义规则，隐藏实现细节

封装 Encapsulation

把属性设为 `private`，通过 `public` 方法访问。

代码块

```
1  class Student {
2      private int age;
3
4      public void setAge(int age) {
5          if (age > 0) {
6              this.age = age;
7          }
8      }
9
10     public int getAge() {
11         return age;
12     }
13 }
```

一句话：

封装就是隐藏内部数据，防止外部随便修改。

继承 Inheritance

子类使用父类的属性和方法，用 `extends`。

代码块

```
1  class Animal {
2      void eat() {
```

```
3         System.out.println("eat");
4     }
5 }
6
7 class Dog extends Animal {
8     void bark() {
9         System.out.println("bark");
10    }
11 }
```

一句话：

继承就是子类复用父类代码，表示 is-a 关系。

例如：

代码块

```
1 Dog is an Animal.
```

多态 Polymorphism

父类引用指向子类对象。

代码块

```
1 Animal a = new Dog();
```

完整例子：

代码块

```
1 class Animal {
2     void sound() {
3         System.out.println("Animal sound");
4     }
5 }
6
7 class Dog extends Animal {
8     void sound() {
9         System.out.println("Dog barks");
10    }
11 }
12
```

```
13 Animal a = new Dog();
14 a.sound();
```

输出：

代码块

```
1 Dog barks
```

一句话：

多态就是同一个父类引用，在运行时根据实际子类对象表现出不同结果。

抽象 Abstraction

只定义规则，不写具体实现。

代码块

```
1 abstract class Shape {
2     abstract double getArea();
3 }
4
5 class Circle extends Shape {
6     double getArea() {
7         return 3.14 * 2 * 2;
8     }
9 }
```

一句话：

抽象就是规定“必须做什么”，具体“怎么做”由子类实现。

最短记忆：

代码块

```
1 封装：private 保护数据
2 继承：extends 复用代码
3 多态：Parent p = new Child()
4 抽象：abstract/interface 定规则
```

